

主要内容

- 链接
- 案例研究：库打桩/插桩 (**Library interpositioning**)

案例研究：库打桩

- 库打桩：一种强大的链接技术，使得程序员可以截获对任意函数的调用
- 打桩可以发生在：
 - 编译时（当编译源码时）
 - 链接时（当将可重定位目标文件静态链接生成可执行目标文件时）
 - 加载/运行时（当可执行目标文件加载进内存、动态链接和执行时）

打桩技术的一些应用

■ 安全Security

- 限制（沙盒）
- 后台加密

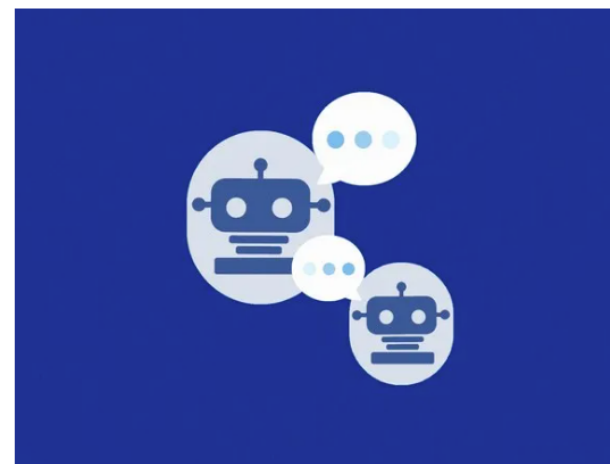
■ 调试

- 2014年两个Facebook (Meta)的工程师用打桩技术调试发现了一个持续一年的bug
- SPDY网络栈中的代码写入到了错误的位置
- 是通过拦截对 Posix 写函数(write, writev, pwrite)的调用解决的

Source: Facebook engineering blog post at <https://code.facebook.com/posts/313033472212144/debugging-file-corruption-on-ios/>

POSTED ON AUGUST 12, 2014 TO IOS

Debugging file corruption on iOS



By Slobodan Predolac, Nicolas Spiegelberg

打桩技术的一些应用

■ 监控与侧写 (**profiling**)

- 对函数调用次数计数
- 描述对函数调用的地点和参数
- 跟踪 `malloc`
 - 发现内存泄漏
 - 生成访存地址跟踪序列

程序示例

```
#include <stdio.h>
#include <malloc.h>

int main()
{
    int *p = malloc(32);
    free(p);
    return(0);
}
int.c
```

- 目标：在不中断程序执行和修改源代码的情况下，跟踪分配和释放块的地址和大小
- 三种解决方法：在编译时、链接时和加载/运行时对库函数 **malloc** 和 **free** 打桩

编译时打桩

```
#ifdef COMPILETIME
#include <stdio.h>
#include <malloc.h>

/* malloc wrapper function */
void *mymalloc(size_t size)
{
    void *ptr = malloc(size);
    printf("malloc(%d)=%p\n",
        (int)size, ptr);
    return ptr;
}

/* free wrapper function */
void myfree(void *ptr)
{
    free(ptr);
    printf("free(%p)\n", ptr);
}
#endif
```

mymalloc.c

编译时打桩

```
#define malloc(size) mymalloc(size)
#define free(ptr) myfree(ptr)
```

```
void *mymalloc(size_t size);
void myfree(void *ptr);
```

malloc.h

```
linux> make intc
```

```
gcc -Wall -DCOMPILETIME -c mymalloc.c
```

```
gcc -Wall -I. -o intc int.c mymalloc.o
```

```
linux> make runc
```

```
./intc
```

```
malloc(32)=0x1edc010
```

```
free(0x1edc010)
```

```
linux>
```

链接时打桩

```
#ifdef LINKTIME
#include <stdio.h>

void *__real_malloc(size_t size);
void __real_free(void *ptr);

/* malloc wrapper function */
void *__wrap_malloc(size_t size)
{
    void *ptr = __real_malloc(size); /* Call libc malloc */
    printf("malloc(%d) = %p\n", (int)size, ptr);
    return ptr;
}

/* free wrapper function */
void __wrap_free(void *ptr)
{
    __real_free(ptr); /* Call libc free */
    printf("free(%p)\n", ptr);
}
#endif
```

mymalloc.c

链接时打桩

```
linux> make intl
gcc -Wall -DLINKTIME -c mymalloc.c
gcc -Wall -c int.c
gcc -Wall -Wl,--wrap,malloc -Wl,--wrap,free -o intl
int.o mymalloc.o
linux> make runl
./intl
malloc(32) = 0x1aa0010
free(0x1aa0010)
linux>
```

- “-Wl”标志是将后面的参数传递给链接器，并用空格替换逗号
- “--wrap,malloc ”参数指示链接器以一种特殊的方式解析引用
 - 对 malloc 的引用应当被解析成 __wrap_malloc
 - 对 __real_malloc 的引用应当被解析成 malloc

加载/运行时打桩

```
#ifdef RUNTIME
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <dlfcn.h>

/* malloc wrapper function */
void *malloc(size_t size)
{
    void *(*mallocp)(size_t size);
    char *error;

    mallocp = dlsym(RTLD_NEXT, "malloc"); /* Get addr of libc malloc */
    if ((error = dlerror()) != NULL) {
        fputs(error, stderr);
        exit(1);
    }
    char *ptr = mallocp(size); /* Call libc malloc */
    printf("malloc(%d) = %p\n", (int)size, ptr);
    return ptr;
}
```

mymalloc.c

加载/运行时打桩

```
/* free wrapper function */
void free(void *ptr)
{
    void (*freep)(void *) = NULL;
    char *error;

    if (!ptr)
        return;

    freep = dlsym(RTLD_NEXT, "free"); /* Get address of libc free */
    if ((error = dlerror()) != NULL) {
        fputs(error, stderr);
        exit(1);
    }
    freep(ptr); /* Call libc free */
    printf("free(%p)\n", ptr);
}
#endif
```

mymalloc.c

加载/运行时打桩

```
linux> make intr
gcc -Wall -DRUNTIME -shared -fpic -o mymalloc.so mymalloc.c -ldl
gcc -Wall -o intr int.c
linux> make runr
(LD_PRELOAD="./mymalloc.so" ./intr)
malloc(32) = 0xe60010
free(0xe60010)
linux>
```

- **LD_PRELOAD** 环境变量告诉动态链接器在解析未被解析的引用时（例如，对 **malloc** 的引用），先在 **mymalloc.so** 中查找

打桩小结

■ 编译时

- 将对 `malloc/free` 的调用宏扩展为对 `mymalloc/myfree` 的调用

■ 链接时

- 使用链接器的技巧做特殊的名字解析
 - `malloc` → `__wrap_malloc`
 - `__real_malloc` → `malloc`

■ 加载/运行时

- 采用动态链接的方法以不同的名字加载库函数 `malloc/free`，实现自定义的 `malloc/free` 版本